

Lab 6: Causal ML for Orange Juice Price Elasticity

Overview

You have been given a dataset consisting of a single file, [oj_large.csv](#). The dataset is a subset of a dataset compiled during a large study by the Chicago Booth School of Business, which collaborated with a local supermarket chain called Dominick's Finer Foods, to study the impact of prices, advertising, and demographics on the sales of a number of products. The dataset we are working with has over 29,000 observations of the price of orange juice and sales for different brands at different Dominick's stores. There is a dataset description here: [oj_dictionary.qmd](#).

The goal of this homework assignment is to use Causal Machine Learning to understand how different demographic factors influence something called the 'price elasticity' of orange juice. You can read more about price elasticity here: [Wikipedia Price Elasticity of Demand](#). If we define demand as sales, the price elasticity of demand is the relationship between a percentage change in sales and a percentage change in price:

$$\epsilon = \frac{\partial(\text{SALES})}{\partial(\text{PRICE})} \frac{\text{PRICE}}{\text{SALES}}$$

This relationship is most natural when expressed in terms of the 'log' transform of both sales and price, as the elasticity becomes the coefficient in a linear regression model relating the two:

$$\log(\text{SALES}) = \epsilon \log(\text{PRICE}) + \text{error terms}$$

The elasticity ϵ can be dependent upon a variety of other factors. It can depend on the price itself (so that we don't get a straight line relationship between the logs), it can depend on the type of product, the demographics of the shoppers and more. The 'EconML' package developed a vignette where they used Causal ML to show that ϵ is a function of income, which is to say that the sales are more sensitive to price in stores where the median income is lower.

You can find that vignette [here](#) and I recommend that you read it and use some code from it as a starting point (it covers more than OJ but it is there): [EconML OJ Vignette](#) . To read more about the package and its applications, see [pywhy EconML](#). The [tutorial for this package](#) is helpful as well.

Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LassoCV
from sklearn.model_selection import train_test_split
from econml.dml import CausalForestDML, LinearDML
from econml.sklearn_extensions.linear_model import WeightedLassoCVWrapper
from sklearn.metrics import r2_score
```

Data

```
np.random.seed(42)

oj = pd.read_csv("https://raw.githubusercontent.com/py-why/EconML/refs/heads/data/datasets/OJ.csv")
print(oj.shape)
print(oj.columns.tolist())
print(oj.head())
```

```
(28947, 17)
['store', 'brand', 'week', 'logmove', 'feat', 'price', 'AGE60', 'EDUC', 'ETHNIC', 'INCOME',
  store      brand  week  logmove  feat  price  AGE60  EDUC  ETHNIC  \
0         2  tropicana   40  9.018695    0   3.87  0.232865  0.248935  0.11428
1         2  tropicana   46  8.723231    0   3.87  0.232865  0.248935  0.11428
2         2  tropicana   47  8.253228    0   3.87  0.232865  0.248935  0.11428
3         2  tropicana   48  8.987197    0   3.87  0.232865  0.248935  0.11428
4         2  tropicana   50  9.093357    0   3.87  0.232865  0.248935  0.11428

      INCOME  HHLARGE  WORKWOM  HVAL150  SSTRDIST  SSTRVOL  CPDIST5  \
0  10.553205  0.103953  0.303585  0.463887  2.110122  1.142857  1.92728
1  10.553205  0.103953  0.303585  0.463887  2.110122  1.142857  1.92728
```

2	10.553205	0.103953	0.303585	0.463887	2.110122	1.142857	1.92728
3	10.553205	0.103953	0.303585	0.463887	2.110122	1.142857	1.92728
4	10.553205	0.103953	0.303585	0.463887	2.110122	1.142857	1.92728

```

CPWVOL5
0 0.376927
1 0.376927
2 0.376927
3 0.376927
4 0.376927

```

Problem 1: Testing on Fake Data

- (a) It is standard practice in Causal Inference to test models on simulated response data based on the original covariates of the dataset before fitting to the original dataset. Use the same selection of confounders as in the original vignette (The ‘W’ matrix), excluding ‘week’, ‘store’, ‘price’, ‘INCOME’, and ‘logmove’, applying One-Hot encoding/dummies to the ‘brand’ variables to incorporate them into ‘W’. Apply the ‘StandardScaler’ to ‘W’. Then put the variable ‘INCOME’ into a matrix called “X” and standardize it. The matrix ‘X’ contains *modifier* variables whose effect on the elasticity will be studied. Then you will simulate a relationship between your confounders and the price, you can use code like this: ‘T_sim = 0.8 + W[:, support] @ coefs_T + noise’, where ‘support’ is sparse (most entries are 0, the rest are 1) and ‘coefs_T’ is random (the 0.8 is just for scale). Look to the simulation code earlier in the ‘EconML’ vignette for inspiration.

```

# build W
exclude = ["week", "store", "price", "INCOME", "logmove", "brand"]
W_cols = [c for c in oj.columns if c not in exclude + ["brand"]]

# one hot encode and add to w
brand_dummies = pd.get_dummies(oj["brand"], prefix="brand", drop_first=False)
W_raw = pd.concat([oj[W_cols], brand_dummies], axis=1).astype(float)

scaler_W = StandardScaler()
W = scaler_W.fit_transform(W_raw)

# build X
X_raw = oj[["INCOME"]].values.astype(float)
scaler_X = StandardScaler()
X = scaler_X.fit_transform(X_raw)

```

Simulate T

```

n, p_w = W.shape
n_support = 5
support = np.random.choice(p_w, n_support, replace=False)

coefs_T = np.random.uniform(0.5, 1.5, size=n_support)
noise_T = np.random.normal(0, 0.5, size=n)

T_sim = 0.8 + W[:, support] @ coefs_T + noise_T

print(f"W shape: {W.shape}")
print(f"X shape: {X.shape}")
print(f"T_sim mean: {T_sim.mean():.3f}, std: {T_sim.std():.3f}")

```

```

W shape: (28947, 14)
X shape: (28947, 1)
T_sim mean: 0.802, std: 2.094

```

- (b) Now, simulate the values of the ‘logmove’ (in a matrix called ‘Y_sim’) using your ‘T_sim’, your confounders ‘W’, and your modifier ‘X’. Make the relationship between ‘Y_sim’, ‘T_sim’, and ‘X’ nonlinear using something like this: ‘Y_sim = (-2.5 * np.tanh(2.0*X))*T_sim + W[:, support] @ coefs_Y + noise’, where ‘coefs_Y’ is random (we are using the same ‘support’ in both simulations).

```

coefs_Y = np.random.uniform(0.5, 1.5, size=n_support)
noise_Y = np.random.normal(0, 0.5, size=n)

Y_sim = (-2.5 * np.tanh(2.0 * X.ravel())) * T_sim + \
        W[:, support] @ coefs_Y + noise_Y

true_cate = -2.5 * np.tanh(2.0 * X.ravel())

print(f"Y_sim mean: {Y_sim.mean():.3f}")
print(f"True CATE range: [{true_cate.min():.3f}, {true_cate.max():.3f}]")
print(f"True ATE: {true_cate.mean():.4f}")

```

```

Y_sim mean: -1.634
True CATE range: [-2.499, 2.500]
True ATE: -0.1621

```

- (c) Using the code from the vignette, fit a Causal Forest (‘CausalForestDML’) and a linear model (‘LinearDML’) to the simulated data. For both models, plot the predicted

elasticity as a function of the 'INCOME', showing the confidence intervals and the real relationship. Also plot the predicted elasticity and the true elasticity for each simulated observation. Report the true and estimated ATE with confidence intervals. Comment on the performance of both models on the simulated data.

Fit Causal Model

```
cf = CausalForestDML(  
    model_y=WeightedLassoCVWrapper(cv=3),  
    model_t=WeightedLassoCVWrapper(cv=3),  
    n_estimators=500,  
    min_samples_leaf=50,  
    max_depth=20,  
    max_samples=0.04 / 2,  
    random_state=42  
)  
cf.fit(Y_sim, T_sim, X=X, W=W, cache_values=True)
```

C:\Users\jpfer\python-versions\py3103\lib\site-packages\econml\sklearn_extensions\model_selection

Model <econml.sklearn_extensions.linear_model.WeightedLassoCVWrapper object at 0x00000255BA63...>
default cv attribute, which will be ignored

C:\Users\jpfer\python-versions\py3103\lib\site-packages\econml\sklearn_extensions\model_selection

Model <econml.sklearn_extensions.linear_model.WeightedLassoCVWrapper object at 0x00000255BA63...>
default cv attribute, which will be ignored

<econml.dml.causal_forest.CausalForestDML at 0x25590a36800>

Fit linear model

```
lm = LinearDML(  
    model_y=WeightedLassoCVWrapper(cv=3),  
    model_t=WeightedLassoCVWrapper(cv=3),  
    random_state=42,  
    cv=5  
)  
lm.fit(Y_sim, T_sim, X=X, W=W)
```

C:\Users\jpfer\python-versions\py3103\lib\site-packages\econml\sklearn_extensions\model_sele

Model <econml.sklearn_extensions.linear_model.WeightedLassoCVWrapper object at 0x00000255909
default cv attribute, which will be ignored

C:\Users\jpfer\python-versions\py3103\lib\site-packages\econml\sklearn_extensions\model_sele

Model <econml.sklearn_extensions.linear_model.WeightedLassoCVWrapper object at 0x00000255909
default cv attribute, which will be ignored

<econml.dml.dml.LinearDML at 0x255ba5c6cb0>

Plot Elasticity vs Income

```
X_test_grid = np.linspace(X.min(), X.max(), 200).reshape(-1, 1)
true_effect_grid = -2.5 * np.tanh(2.0 * X_test_grid.ravel())

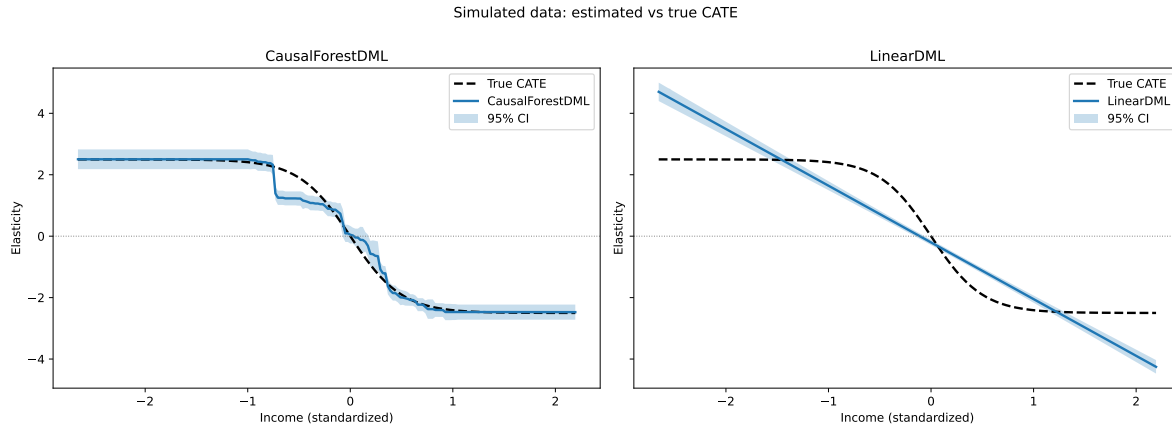
cf_eff = cf.const_marginal_effect(X_test_grid)
cf_lb, cf_ub = cf.const_marginal_effect_interval(X_test_grid, alpha=0.05)

lm_eff = lm.const_marginal_effect(X_test_grid)
lm_lb, lm_ub = lm.const_marginal_effect_interval(X_test_grid, alpha=0.05)

fig, axes = plt.subplots(1, 2, figsize=(14, 5), sharey=True)

for ax, name, eff, lb, ub in [
    (axes[0], "CausalForestDML", cf_eff.ravel(), cf_lb.ravel(), cf_ub.ravel()),
    (axes[1], "LinearDML", lm_eff.ravel(), lm_lb.ravel(), lm_ub.ravel()),
]:
    ax.plot(X_test_grid, true_effect_grid, "k--", lw=2, label="True CATE")
    ax.plot(X_test_grid, eff, lw=2, label=name)
    ax.fill_between(X_test_grid.ravel(), lb, ub, alpha=0.25, label="95% CI")
    ax.axhline(0, color="gray", lw=0.8, linestyle=":")
    ax.set_xlabel("Income (standardized)")
    ax.set_ylabel("Elasticity")
    ax.set_title(name)
    ax.legend()

plt.suptitle("Simulated data: estimated vs true CATE", y=1.01)
plt.tight_layout()
plt.show()
```

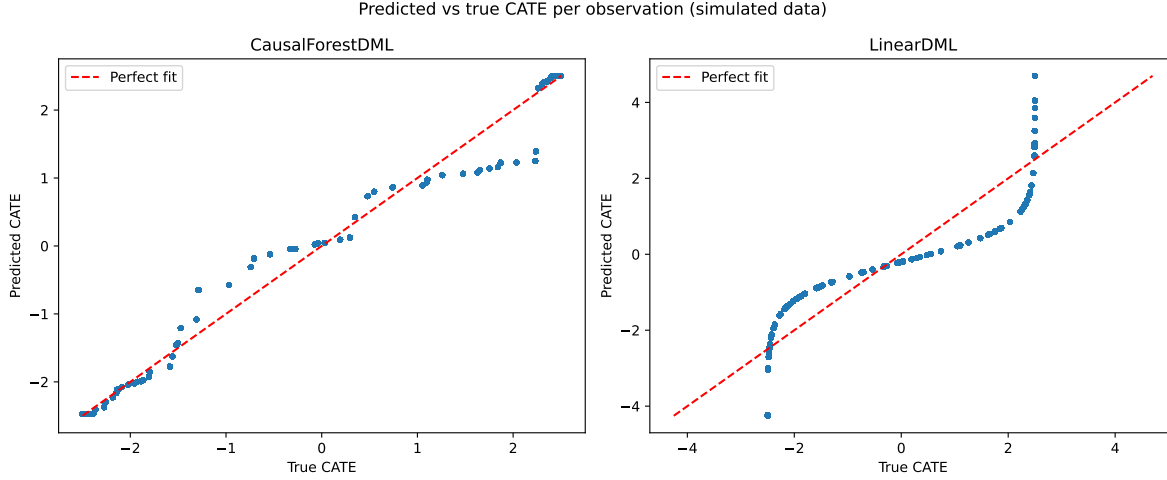


Plot Predicted Elasticity vs True Elasticity for Causal and Linear Models

```
cf_obs_eff = cf.effect(X)
lm_obs_eff = lm.effect(X)
true_cate = -2.5 * np.tanh(2.0 * X.ravel())

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, name, pred in [(axes[0], "CausalForestDML", cf_obs_eff),
                       (axes[1], "LinearDML", lm_obs_eff)]:
    ax.scatter(true_cate, pred, alpha=0.15, s=10)
    mn = min(true_cate.min(), pred.min())
    mx = max(true_cate.max(), pred.max())
    ax.plot([mn, mx], [mn, mx], "r--", lw=1.5, label="Perfect fit")
    ax.set_xlabel("True CATE")
    ax.set_ylabel("Predicted CATE")
    ax.set_title(name)
    ax.legend()

plt.suptitle("Predicted vs true CATE per observation (simulated data)")
plt.tight_layout()
plt.show()
```



On the simulated data, both models are able to recover the general negative relationship between price and sales, telling us that higher prices reduce demand. However, the true elasticity was generated as a nonlinear function of income, which creates clear differences in model performance. The CausalForestDML model closely follows the true nonlinear relationship between income and elasticity, especially at the extremes of the income distribution. On the other hand, the LinearDML model captures the overall trend but fails to reproduce the nonlinear shape, instead producing a more linear approximation. This is also visible in the predicted vs. true elasticity plots. The Causal Forest predictions are more tightly clustered around the 45-degree line, indicating a better fit at the observation level. The Linear model shows more systematic deviations, particularly in regions where the true relationship is highly nonlinear. This confirms that the simulation test behaves as expected.

ATE with Confidence Intervals

```
cf_ate_inf = cf.ate_interval(X, alpha=0.05)
lm_ate_inf = lm.ate_interval(X, alpha=0.05)

print(f"True ATE:                {true_cate.mean():.4f}")
print(f"CausalForest ATE:        {cf.ate(X).item():.4f}  ")
      f"95% CI: [{cf_ate_inf[0].item():.4f}, {cf_ate_inf[1].item():.4f}]")
print(f"LinearDML ATE:           {lm.ate(X).item():.4f}  ")
      f"95% CI: [{lm_ate_inf[0].item():.4f}, {lm_ate_inf[1].item():.4f}]")
```

```
True ATE:                -0.1621
CausalForest ATE:        -0.1980  95% CI: [-0.4766, 0.0806]
LinearDML ATE:           -0.2028  95% CI: [-0.2888, -0.1169]
```

The true ATE is around -0.162, and both models produce estimates that are reasonably close to this value. However, there is a clear difference in the confidence intervals. The LinearDML

model produces a narrower confidence interval that does not include zero, suggesting a statistically significant negative effect. In contrast, the CausalForestDML model produces a wider interval that includes zero, so there is a greater uncertainty in its estimate. The Linear model, while more precise, may be overly restrictive and miss important heterogeneity.

The CausalForestDML model performs better in recovering the true heterogeneous treatment effects, particularly due to its ability to capture nonlinear relationships. While the LinearDML model gives more precise estimates of the average treatment effect, it fails to accurately model the variation in elasticity across income levels.

Problem 2: Checking for Overlap

- (a) In order for Causal ML to be successful, there needs to be variation in the treatment variable for all combinations of the confounder variables. For a continuous treatment, it is important that there is residual variation left-over after the Causal Forest predicts the treatment using confounders. Keeping with the structure of the original vignette (same definition of ‘W’, ‘X’, ‘Y’, and ‘T’), use ‘LassoCV’ to predict T using W . Calculate and report the R^2 . Does this value of R^2 support the suitability of this dataset for Causal Inference?

```
# real outcome and treatment
Y_real = oj["logmove"].values
T_real = np.log(oj["price"]).values

scaler_W2 = StandardScaler()
W1_real = scaler_W2.fit_transform(
    oj[[c for c in oj.columns
        if c not in ["price", "logmove", "brand", "week", "store", "INCOME"]]].values
)
W2_real = pd.get_dummies(oj[["brand"]]).values
W_real = np.concatenate([W1_real, W2_real], axis=1)
X_real = oj[["INCOME"]].values

# Lasso to predict T from W
lasso_cv = LassoCV(cv=5, random_state=42, n_jobs=1)
lasso_cv.fit(W_real, T_real)
T_pred = lasso_cv.predict(W_real)
r2 = r2_score(T_real, T_pred)

print(f"LassoCV R2 (T ~ W): {r2:.4f}")
print(f"Residual variance fraction remaining: {1 - r2:.4f}")
```

LassoCV R^2 ($T \sim W$): 0.5740
Residual variance fraction remaining: 0.4260

The LassoCV model explains about 57.4% of the variation in the treatment using the confounders, leaving 42.6% as residual variation. Since the R^2 is not close to 1, there is still meaningful variation in price after accounting for W . This suggests that the overlap assumption is reasonably satisfied and the dataset is suitable for causal analysis.

Problem 3: Fitting and Interpreting the Model

- (a) Perform a train-test split on the data. Repeat the fit in the vignette to the training set (you can copy their code with suitable modifications to make it work) using a ‘CausalForestDML’ model to learn the effect of income on price elasticity. Plot the price elasticity versus income with confidence intervals. Calculate the average treatment effect and confidence intervals.

```
n_real = len(Y_real)
idx = np.arange(n_real)
idx_train, idx_test = train_test_split(idx, test_size=0.2, random_state=42)

Y_train, Y_test = Y_real[idx_train], Y_real[idx_test]
T_train, T_test = T_real[idx_train], T_real[idx_test]
W_train, W_test = W_real[idx_train], W_real[idx_test]
X_train, X_test = X_real[idx_train], X_real[idx_test]

cf_real = CausalForestDML(
    model_y=WeightedLassoCVWrapper(cv=3),
    model_t=WeightedLassoCVWrapper(cv=3),
    n_estimators=1000,
    min_samples_leaf=50,
    max_depth=20,
    max_samples=0.04 / 2,
    random_state=42
)
cf_real.fit(Y_train, T_train, X=X_train, W=W_train, cache_values=True)

min_income = 10.0
max_income = 11.1
delta = (max_income - min_income) / 100
X_test_grid = np.arange(min_income, max_income + delta - 0.001, delta).reshape(-1, 1)
```

```

# CATE curve and CIs
cf_effects      = cf_real.const_marginal_effect(X_test_grid)
cf_lb, cf_ub    = cf_real.const_marginal_effect_interval(X_test_grid, alpha=0.05)

# plot
plt.figure(figsize=(8, 5))
plt.plot(X_test_grid.flatten(), cf_effects, label="OJ Elasticity")
plt.fill_between(X_test_grid.flatten(), cf_lb.ravel(), cf_ub.ravel(),
                 alpha=0.3, label="95% CI")
plt.axhline(0, color="gray", lw=0.8, linestyle=":")
plt.xlabel(r"$\log$(Income)")
plt.ylabel("Orange Juice Elasticity")
plt.title("OJ Price Elasticity vs Income: CausalForestDML")
plt.legend()
plt.tight_layout()
plt.show()

# ATE with CIs
cf_ate          = cf_real.ate(X_train)
cf_ate_lb, cf_ate_ub = cf_real.ate_interval(X_train, alpha=0.05)
print(f"CausalForest ATE: {cf_ate.item():.4f}  "
      f"95% CI: [{cf_ate_lb.item():.4f}, {cf_ate_ub.item():.4f}]")

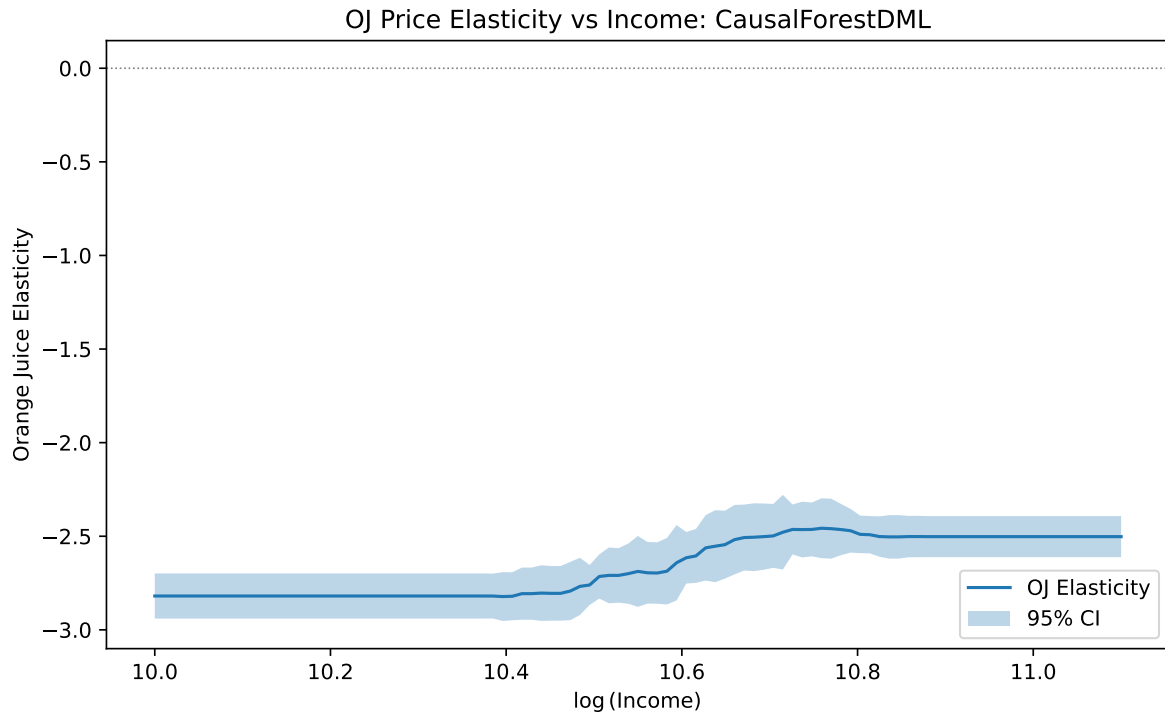
```

C:\Users\jpfer\python-versions\py3103\lib\site-packages\econml\sklearn_extensions\model_sele

Model <econml.sklearn_extensions.linear_model.WeightedLassoCVWrapper object at 0x00000255C28
default cv attribute, which will be ignored

C:\Users\jpfer\python-versions\py3103\lib\site-packages\econml\sklearn_extensions\model_sele

Model <econml.sklearn_extensions.linear_model.WeightedLassoCVWrapper object at 0x00000255C28
default cv attribute, which will be ignored



CausalForest ATE: -2.6193 95% CI: [-2.7602, -2.4784]

- (b) Compute the R-score on the testing/validation set to determine the strength of the heterogeneity. What is your interpretation of the R-score value? Fit a 'LinearDML' model in the same manner and compare the R-score to the 'CausalForestDML'. Is either model noticeably better?

R-score and LinearDML Comparison

```
cf_rscore = cf_real.score(Y_test, T_test, X=X_test, W=W_test)
print(f"CausalForestDML R-score (test): {cf_rscore:.4f}")

# fit LinearDML
lm_real = LinearDML(
    model_y=WeightedLassoCVWrapper(cv=3),
    model_t=WeightedLassoCVWrapper(cv=3),
    random_state=42,
    cv=5
)
lm_real.fit(Y_train, T_train, X=X_train, W=W_train, cache_values=True)
```

```

lm_rscore = lm_real.score(Y_test, T_test, X=X_test, W=W_test)
print(f"LinearDML R-score (test):          {lm_rscore:.4f}")

# plot LinearDML elasticity curve
lm_effects = lm_real.const_marginal_effect(X_test_grid)
lm_lb, lm_ub = lm_real.const_marginal_effect_interval(X_test_grid, alpha=0.05)

fig, axes = plt.subplots(1, 2, figsize=(14, 5), sharey=True)
for ax, name, eff, lb, ub in [
    (axes[0], f"CausalForestDML (R²={cf_rscore:.3f})",
     cf_effects.ravel(), cf_lb.ravel(), cf_ub.ravel()),
    (axes[1], f"LinearDML (R²={lm_rscore:.3f})",
     lm_effects.ravel(), lm_lb.ravel(), lm_ub.ravel()),
]:
    ax.plot(X_test_grid.flatten(), eff, lw=2, label="OJ Elasticity")
    ax.fill_between(X_test_grid.flatten(), lb, ub, alpha=0.3, label="95% CI")
    ax.axhline(0, color="gray", lw=0.8, linestyle=":")
    ax.set_xlabel(r"$\log$(Income)")
    ax.set_ylabel("Elasticity")
    ax.set_title(name)
    ax.legend()

plt.suptitle("OJ Elasticity vs Income: CausalForest vs LinearDML")
plt.tight_layout()
plt.show()

```

CausalForestDML R-score (test): 0.4524

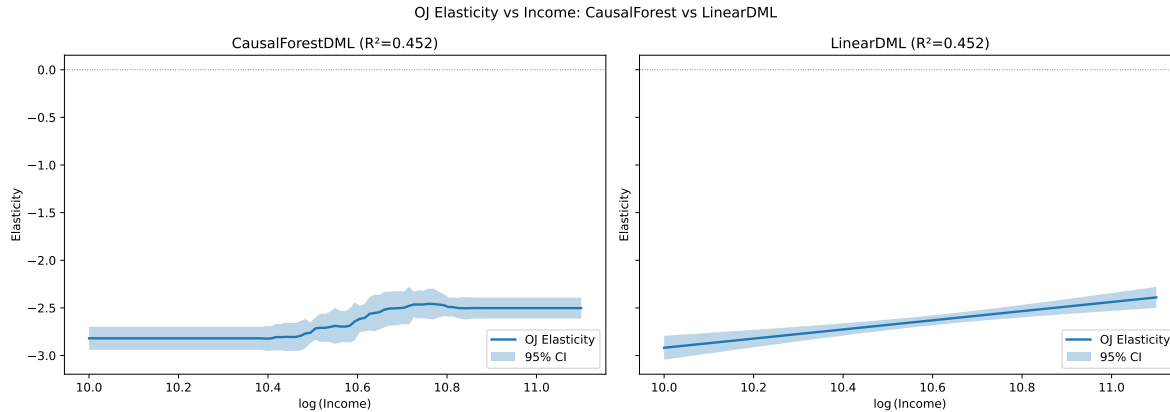
C:\Users\jpfer\python-versions\py3103\lib\site-packages\econml\sklearn_extensions\model_selection

Model <econml.sklearn_extensions.linear_model.WeightedLassoCVWrapper object at 0x00000255C28...
default cv attribute, which will be ignored

C:\Users\jpfer\python-versions\py3103\lib\site-packages\econml\sklearn_extensions\model_selection

Model <econml.sklearn_extensions.linear_model.WeightedLassoCVWrapper object at 0x00000255C28...
default cv attribute, which will be ignored

LinearDML R-score (test): 0.4521



The CausalForestDML model estimated an average price elasticity of -2.6193, with a 95% confidence interval of [-2.7602, -2.4784]. Since the entire interval is below 0, the model suggests a strong negative relationship between price and sales: as orange juice prices increase, sales decrease.

The R-scores for both models are approximately 0.45, indicating a moderate level of treatment-effect heterogeneity captured by the models. This suggests that income explains a meaningful portion of variation in price elasticity, but not all of it. Since the scores are nearly identical, neither model is clearly better at capturing heterogeneity, and the relationship may not be strongly nonlinear in the real data.

- (c) Compute a sensitivity check using the ‘sensitivity_interval’ method of your fit model. This determines how strong an *unobserved confounder* would have to be to change the results of your analysis in a meaningful way. The method recalculates confidence intervals for the *ATE* based on two parameters ‘c_t’ and ‘c_y’, which are the fraction of residual variance explained by the hypothetical confounder for the treatment and the target respectively. One method for determining the range of ‘c_t’ and ‘c_y’ to explore by checking the values of ‘c_t’ and ‘c_y’ for existing confounders. If you were to do this check, you would find that the most important confounder is the ‘feat’ variable (whether the item was advertised that week), and the range should be up to ‘c_t=0.1’ and ‘c_y=0.3’. Compute the sensitivity check for the most extreme scenario, with ‘c_t=0.1’ and ‘c_y=0.3’. What are the resulting confidence intervals for the ATE? Do they contain 0?

```
# sensitivity interval extreme scenario
sens_lb, sens_ub = cf_real.sensitivity_interval(
    c_t=0.1,
    c_y=0.3,
    alpha=0.05
)
```

```

print(f"Sensitivity interval ATE (c_t=0.1, c_y=0.3): "
      f"[{sens_lb.item():.4f}, {sens_ub.item():.4f}]")
print(f"Original ATE CI: [{cf_ate_lb.item():.4f}, {cf_ate_ub.item():.4f}]")
print(f"Contains 0: {sens_lb.item() < 0 < sens_ub.item()}")

```

```

Sensitivity interval ATE (c_t=0.1, c_y=0.3): [-3.3287, -1.9323]
Original ATE CI: [-2.7602, -2.4784]
Contains 0: False

```

The sensitivity interval for the ATE under the extreme scenario $c_t = 0.1$ and $c_y = 0.3$ is $[-3.3287, -1.9323]$. This is wider than the original ATE confidence interval, which was $[-2.7602, -2.4784]$, showing that possible unobserved confounding increases uncertainty. However, the sensitivity interval still does not contain 0, so the estimated negative price elasticity seems robust even under this stronger scenario.

Problem 4: CATE for Brands and Income

- (a) The three brands of orange juice have different price points and are targetted at different customer segments, with ‘dominicks’ as the discount brand, ‘minute.maid’ as the mid-range brand, and ‘tropicana’ as the premium brand. Move the brand variables from confounder matrix W to the modifier matrix X and refit the model. Calculate the feature importances for all the modifiers and plot the elasticity as a function of income for each of the three brands. How do the elasticities differ by brand?

```

# rebuild W with no brand, add brand to X
W1_p4 = scaler_W2.fit_transform(
    oj[[c for c in oj.columns
        if c not in ["price", "logmove", "brand", "week", "store", "INCOME"]]]
)
W_p4 = W1_p4

brand_dummies_p4 = pd.get_dummies(oj["brand"], prefix="brand").values.astype(float)
X_income         = oj[["INCOME"]].values
X_p4              = np.concatenate([X_income, brand_dummies_p4], axis=1)

```

Train/test split using same indices

```

W_train_p4, W_test_p4 = W_p4[idx_train], W_p4[idx_test]
X_train_p4, X_test_p4 = X_p4[idx_train], X_p4[idx_test]

# Fit CausalForestDML
cf_p4 = CausalForestDML(
    model_y=WeightedLassoCVWrapper(cv=3),
    model_t=WeightedLassoCVWrapper(cv=3),
    n_estimators=1000,
    min_samples_leaf=50,
    max_depth=20,
    max_samples=0.04 / 2,
    random_state=42
)
cf_p4.fit(Y_train, T_train, X=X_train_p4, W=W_train_p4, cache_values=True)

# features importance
feature_names = ["INCOME", "brand_dominicks", "brand_minute.maid", "brand_tropicana"]
importances = cf_p4.feature_importances_

imp_series = pd.Series(importances, index=feature_names).sort_values(ascending=False)
print("Feature importances:")
print(imp_series)

imp_series.plot(kind="barh", figsize=(7, 4),
                title="CausalForestDML feature importances (X modifiers)")
plt.xlabel("Importance")
plt.tight_layout()
plt.show()

```

C:\Users\jpfer\python-versions\py3103\lib\site-packages\econml\sklearn_extensions\model_selection

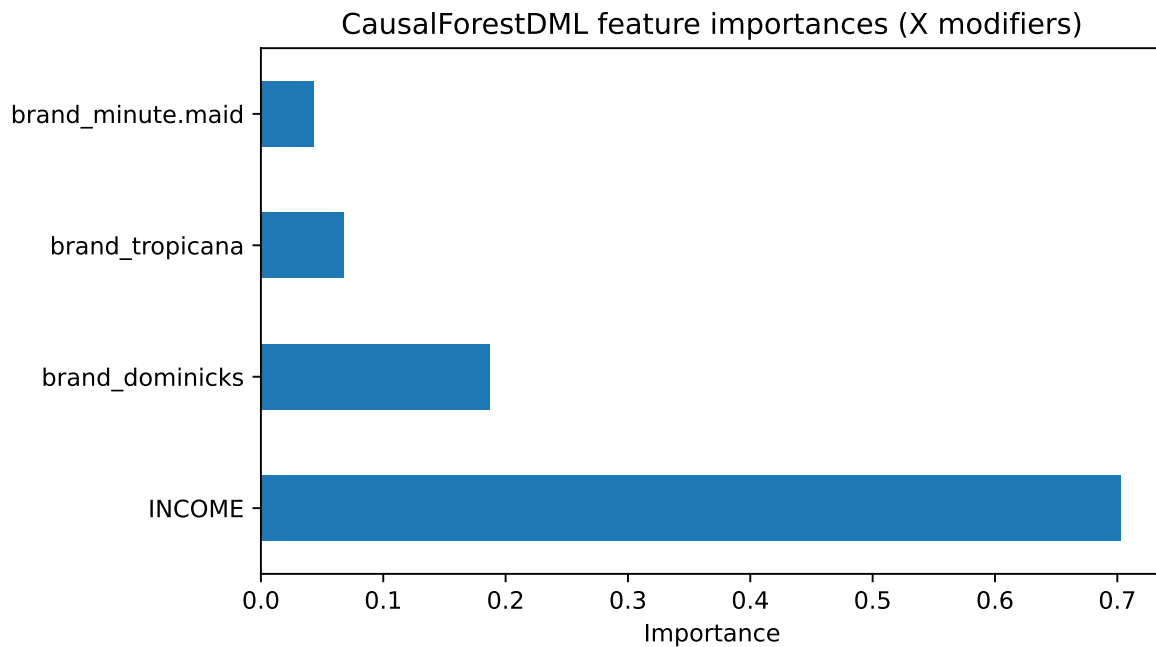
Model <econml.sklearn_extensions.linear_model.WeightedLassoCVWrapper object at 0x00000255C293...
default cv attribute, which will be ignored

C:\Users\jpfer\python-versions\py3103\lib\site-packages\econml\sklearn_extensions\model_selection

Model <econml.sklearn_extensions.linear_model.WeightedLassoCVWrapper object at 0x00000255C293...
default cv attribute, which will be ignored

Feature importances:
INCOME 0.702417

```
brand_dominicks      0.186900
brand_tropicana      0.067930
brand_minute.maid    0.042752
dtype: float64
```



Elasticity vs income for each brand

```
# build x_test_grid for each brand
brands      = ["dominicks", "minute.maid", "tropicana"]
brand_codes = np.eye(3)

income_grid = np.arange(10.0, 11.1 + 0.01 - 0.001, (11.1 - 10.0) / 100)

plt.figure(figsize=(9, 5))
colors = ["steelblue", "coral", "green"]

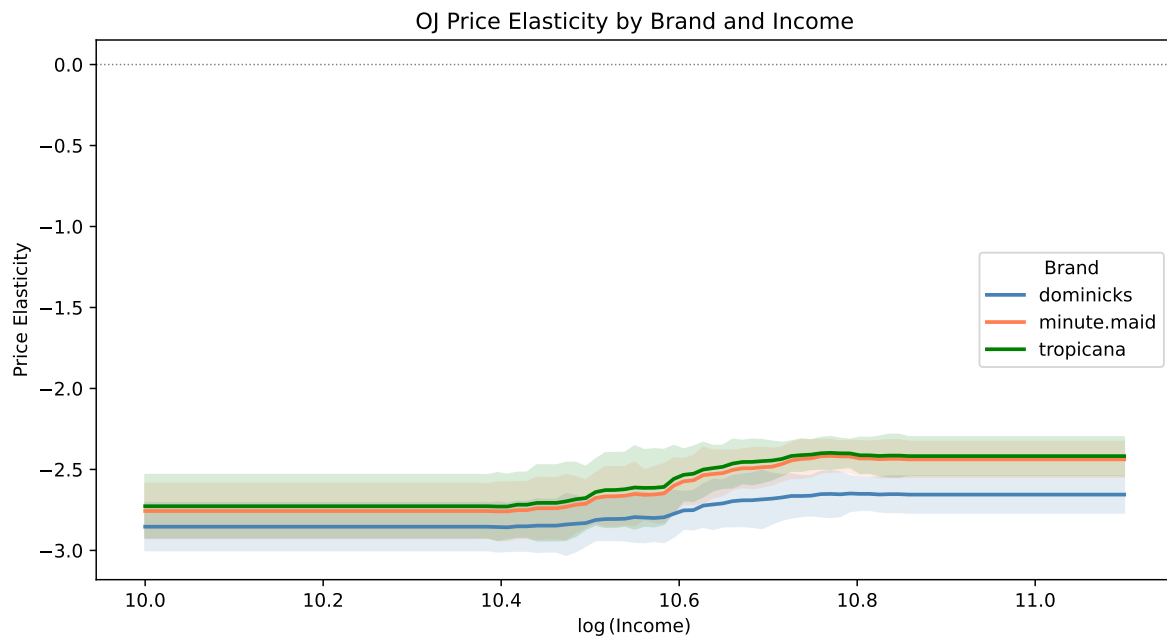
for brand, code, color in zip(brands, brand_codes, colors):
    X_brand_grid = np.column_stack([
        income_grid,
        np.tile(code, (len(income_grid), 1))
    ])
    eff      = cf_p4.const_marginal_effect(X_brand_grid).ravel()
    lb, ub   = cf_p4.const_marginal_effect_interval(X_brand_grid, alpha=0.05)
```

```

plt.plot(income_grid, eff, lw=2, color=color, label=brand)
plt.fill_between(income_grid, lb.ravel(), ub.ravel(), alpha=0.15, color=color)

plt.axhline(0, color="gray", lw=0.8, linestyle=":")
plt.xlabel(r"$\log$(Income)")
plt.ylabel("Price Elasticity")
plt.title("OJ Price Elasticity by Brand and Income")
plt.legend(title="Brand")
plt.tight_layout()
plt.show()

```



ATE per brand

```

print("\nATE by brand:")
for brand, code in zip(brands, brand_codes):
    X_brand_all = np.column_stack([
        X_train_p4[:, 0],          # income of training obs
        np.tile(code, (len(X_train_p4), 1))
    ])
    ate = cf_p4.ate(X_brand_all)
    lb, ub = cf_p4.ate_interval(X_brand_all, alpha=0.05)
    print(f"  {brand:<15} ATE={ate.item():.4f}  95% CI: [{lb.item():.4f}, {ub.item():.4f}]" )

```

ATE by brand:

dominicks	ATE=-2.7415	95% CI: [-2.8781, -2.6049]
minute.maid	ATE=-2.5710	95% CI: [-2.7131, -2.4288]
tropicana	ATE=-2.5416	95% CI: [-2.7053, -2.3780]

Dominick's has the most negative elasticity, which means that its customers are most price-sensitive, especially at lower incomes. Tropicana has the least negative elasticity, so its buyers are less responsive to price changes. The plot shows that elasticity becomes less negative at a higher income for all brands, which makes sense since it is expected that budget-constrained consumers will be more responsive to price changes.